Parallelizing Quantized LLM Inference Under a Fixed Numerical Reference

Chengzhan Li[a,*]

[a]University of Electronic Science and Technology of China, Chengdu, China

ARTICLE INFO

Keywords:
Large language model inference
Parallel execution
Quantization
Numerical reproducibility
KV cache
Speculative decoding

ABSTRACT

Batching known queries can change the outputs and persistent state of quantized language-model inference, even when causal token dependencies are unchanged. We present a parallel execution construction that retains each group's numerical path while reorganizing execution across groups. The construction separates the physical KV range required by attention from the arithmetic grouping used by quantized projections. It places necessary range cuts inside attention, exposes compatible arithmetic groups to the backend, and returns owned output rows with the successor state required by the consumer. A conditional composition argument identifies the structural invariants and backend arithmetic conditions that connect these steps. In an A5000 Qwen3-32B deployment, an instance with one-column arithmetic groups and whole-call cuts reduces target-only resident-service time by 1.51% relative to a strong backend scalar-loop control that already shares activation quantization (95% paired-root interval: 1.23–1.79%). Factorial and boundary experiments separate the effects of projection packing and local cuts, while application measurements account for the work surrounding them. The results explain why a large saving at a controlled boundary can yield little service benefit: compatible grouping opportunities must occur often enough, and the work they save must exceed the costs of views, reconstruction and additional speculative computation.

1. Introduction

Consider four known query tokens at positions 765–768. Batching them exposes parallel work and weight reuse. At a KV-cache padding boundary, a wider call can enlarge earlier queries' physical attention ranges. The causal mask hides the same future tokens, yet output bytes can change. Query width can also select a different activation quantizer or dot-product specialization. The inputs are already available as a prompt suffix or a seed followed by draft proposals; grouping them requires no prediction of a causally unknown token.

Existing software can depend on a numerical path. A regression test compares complete output vectors; an exact replay resumes stored cache state; a replacement component hands state to an unchanged consumer. These settings fix the reference before optimization. We ask: *what parallel execution remains available under that constraint?*

Several approaches already provide deterministic inference. Batch-invariant kernels make numerical results insensitive to batch composition [1]. Fixed scheduling makes selected execution shapes repeatable [2]. Verified speculation uses a consistent verifier to repair a fast path and its KV state [3]. Arithmetic reconstruction also shows that output tiling and pipelining can change while a GPU kernel retains its per-output computation [4]. The question here is how to apply that execution freedom within a quantized decoder whose query-specific numerical path is already chosen. We retain that path while changing the placement of attention cuts and the organization of independent projection groups.

The construction separates two grouping decisions. A *physical run* contains adjacent queries that can share the reference's attention range and descriptor. An *arithmetic group* retains a projection's column width, activation quantization and reduction organization. The boundaries can differ: attention may need a cut inside a group of projections that remains compatible when evaluated together. Keeping that cut inside attention preserves wider surrounding work. Exposing several compatible arithmetic groups through an outer tensor dimension provides another source of parallelism. The executor then restores the logical output order and commits the accepted cache prefix together with the consumer's token state (Fig. 1).

This organization reuses existing small-column kernels, packing, weight broadcasting and fixed reductions. Its contribution is the construction that combines their arithmetic requirements with physical attention ranges and downstream output/state obligations. Structural invariants make the row mapping and cut placement explicit; backend conditions state what must hold for the arithmetic; qualification checks those conditions' consequences in the evaluated implementations. A conditional composition argument connects the resulting operations to subsequent decoder calls.

The experiments separate the method's two mechanisms and the costs around them. The 32B mixed-quantization instance keeps one-column arithmetic and whole-call cuts, isolating independent-group scheduling. Against a backend scalar loop that already shares

*Corresponding author.
Email address: chengzhan.li@std.uestc.edu.cn (C. Li)

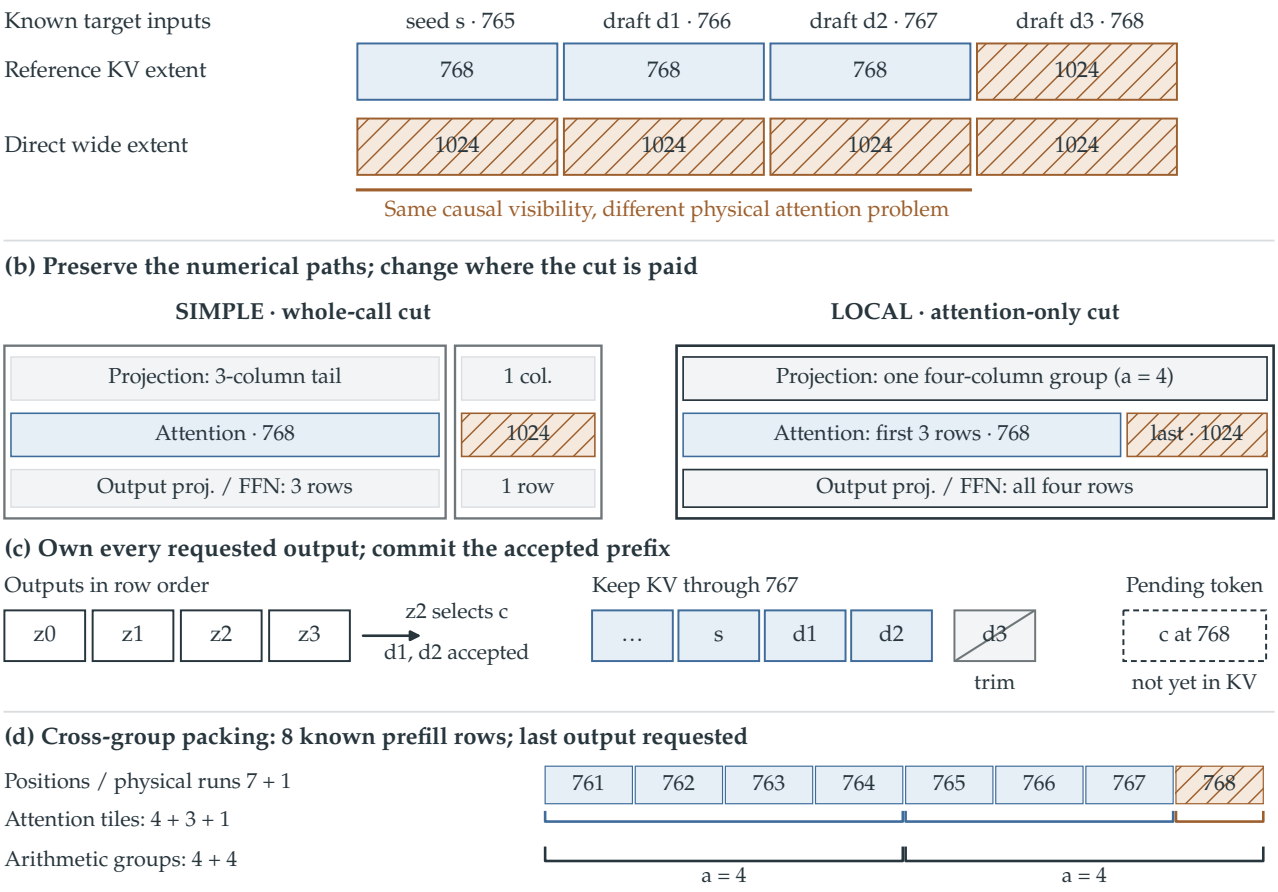


Figure 1: Retain each group's numerical path while reorganizing execution across groups. (a) Four known queries cross a physical KV boundary. (b) Whole-call cuts divide surrounding projections; local cuts retain one four-column projection group while splitting attention. The backend must preserve the reference for each employed one- to four-column path. (c) Owned outputs retain row order; accepting two proposals keeps KV through position 767, while the correction at 768 remains pending. (d) A separate eight-query prefill illustrates two four-column arithmetic groups and attention tiles of four, three and one. Conceptual diagram prepared with ChatGPT (OpenAI, GPT-6 Astra Pro) assistance; editable drawing code is supplied.

activation quantization, it reduces A5000 target-only resident-service time by 1.51% (95% paired-root interval: 1.23–1.79%). Q8 factorials independently vary four-column projection packing and whole versus local attention cuts. Controlled Vulkan appends measure the benefit of crossing a physical boundary; natural requests measure how often that opportunity occurs. Width experiments then show how cheaper equal-work verification can coexist with higher service time when deeper speculation creates extra target and draft work.

The paper contributes:

1. A construction for fixed-reference parallel execution that separates physical and arithmetic groups, locates attention cuts, and preserves requested outputs and successor state.
2. Concrete scalar-column and four-column CUDA instances, together with a Vulkan local-cut study, that connect the construction to existing quantized kernels and fair controls.
3. An explanation of performance through grouping opportunities and their costs, supported by strong-control comparisons, component attribution, application measurements and negative cases.

2. Parallel execution with a fixed reference

2.1. Keep each numerical path, reorganize its execution

Consider four known queries at positions 765–768. With a 256-entry KV padding unit, their scalar-reference physical attention extents are $[768, 768, 768, 1024]$. A direct four-row call can instead expose 1024 entries to every query. The causal mask still hides the same future tokens, but the attention kernel receives a different physical problem. In the captured CUDA case, the visible operands and probability

prefixes agree and the added probabilities are zero; changing the physical extent reproduces the different attention–value outputs [5].

Figure 1 follows these same four queries through the construction. SIMPLE cuts the whole decoder call into three rows and one row. LOCAL keeps one four-row decoder graph, cuts attention into the same $3+1$ groups, and joins the attention results in row order. Projections and feed-forward work remain outside that local cut. For the Q8 implementation, the four-row call contains *one* full arithmetic group of width $a = 4$. Its whole-call counterpart uses admitted three-column and one-column tails. The backend must preserve the reference arithmetic for all these widths; moving the cut alone supplies no such guarantee.

To show cross-group packing, the figure separately extends the example to eight known prompt rows at positions 761–768. Their physical runs have lengths $7+1$, the local attention tiles have lengths $4+3+1$, and the projections have two complete arithmetic groups, $4+4$. A $[K, 4, 2]$ view exposes those two groups to the backend without replacing their inner numerical paths. This is an illustrative wider prefill call with last-row output, not an eight-row call to the four-output verifier.

Known queries also occur in speculative verification. The four inputs can be an already emitted seed s and draft proposals d_1, d_2, d_3. They are known when the target call begins; the causal mask defines the prefix for each target prediction. The method parallelizes evaluation of these supplied prefixes. The separate drafting and acceptance procedure determines which proposals become the actual continuation.

2.2. Reference, outputs and successor state

The deployment reference R_d fixes model and tokenizer bytes, input formatting, numerical modules, placement, cache types and capacity, prompt production, requested outputs, and sampling and stopping rules. Let O contain the row indices whose complete binary32 vocabulary vectors the consumer requests. At the relevant phase and commit boundaries, compatibility requires

$$\begin{aligned} z_i^F &\overset{\text{bits}}{=} z_i^{R_d}, \quad i \in O, \\ \mathcal{K}^F &\overset{\text{bits}}{=} \mathcal{K}^{R_d}, \qquad s^F = s^{R_d}. \end{aligned} \tag{1}$$

Here $\mathcal{K}$ is active target KV in canonical logical order; s contains positions, the retained materialized prefix, any emitted but unmaterialized token, and termination status. Unused cache storage lies outside the relation. A speculative row is compared with the reference following that same supplied prefix; an autonomous reference separately checks emitted tokens and stopping.

The mathematical set O makes the consumer obligation explicit. The evaluated phase frontend exposes *all-row* and *last-row* modes, not an arbitrary sparse-output API. The released Q8 verifier reserves four outputs, so all-row requests have $q \le 4$, while last-row prompt calls can be wider. The 32B profile declares its own reserve. A consumer excludes an output before execution, rather than after observing a discrepancy.

Returning to the four-query example, suppose d_1 and d_2 are accepted and d_3 is rejected. The consumer needs the prediction from row 2, at position 767, to emit a correction c. The executor retains KV positions 0–767, removes the tentative entry at 768, and records c as pending at that position. The next call materializes it. Requested vectors $z_0, \dots, z_3$ are copied into row-owned storage before a later native call can reuse the output buffer. Thus the accepted row remains available even when a later computed row is discarded. Generation can terminate while an emitted token is still pending.

2.3. Constructing the two groupings

The construction takes ordered known rows, the incoming state, a requested-output mode, and a backend profile specifying the admitted arithmetic and attention widths. For a dense nonwrapping append, the scalar-reference extent at zero-based position p_i is

$$e_i = \min\left(C, \max\left(P, P\left\lceil \frac{p_i + 1}{P} \right\rceil\right)\right), \tag{2}$$

where P is native KV padding and C is capacity. The native adapter obtains the metadata and checks the dense append mapping. A phase with a different fixed producer schedule retains that schedule's requirements.

A *physical run* joins adjacent rows with equal extent and compatible attention descriptors. Equal extent alone is insufficient: Q/K/V types, head geometry, V layout and dispatch conditions also matter. Runs are tiled to the admitted attention width. SIMPLE places run boundaries between decoder calls; LOCAL uses restricted Q, K, V and mask views within each wider call and concatenates the results in logical order.

An *arithmetic group* retains the admitted inner column width a. For a projection input $X \in \mathbb{R}^{K \times q}$, separate a possible tail before reshaping the complete groups:

$$\begin{aligned} X[K, q] &= [X_{\text{full}}[K, ag]\ X_{\text{tail}}[K, r]], \\ X_{\text{full}}[K, ag] &\longrightarrow X_{\text{full}}[K, a, g], \\ q &= ag + r, \qquad 0 \le r < a. \end{aligned} \tag{3}$$

Weights are broadcast across the outer group dimension; outputs are reshaped back to column order. A tail follows its separately admitted width. The backend profile fixes quantization and reduction as well as shape.

Captured CUDA replays show why these distinctions matter. Across eight Q8 operator classes in each of Qwen3-0.6B and Qwen3-8B, native widths two and four match concatenated scalar outputs, whereas widths eight, sixteen and thirty-two differ. Keeping

four-column inner groups and packing them along an outer dimension restores complete operator-output agreement at all three wider widths [5]. In the profiled 8B operators, the four- and eight-column kernels use the same activation quantizer but different reduction-warp counts; wider calls select matrix–matrix kernels. The supplementary mechanism table gives the captured-input and dispatch scope. Preserving inner arithmetic, rather than width alone, creates the freedom to reorganize groups.

Construction. The following procedure separates planning, numerical execution and consumer commit. H denotes physical runs, A_c arithmetic groups for call c, and h the admitted attention tile width.

> **Input:** known rows Q, incoming state S, requested rows O from a supported output mode, a reference profile including (a, h), and SIMPLE or LOCAL placement.
>
> 1. Derive each row's extent and descriptor from the bound reference; form adjacent compatible runs H.
> 2. Form outer calls: split at H for SIMPLE; retain wider calls for LOCAL. Respect the native call-width limit in both cases.
> 3. Build each call's usual decoder layer sequence. Partition each eligible projection into A_c of width a and an admitted tail; expose complete groups as outer channels.
> 4. At each attention node, use intersections of c with H, tiled to width h. Join tile outputs by original row index and continue the decoder graph.
> 5. Synchronize each call and copy its requested vectors to owned slots $Y_i \leftarrow z_i$, $i \in O$.
> 6. Let the consumer choose the retained prefix length m, pending token u (if any), and stop status from Y.
> 7. Trim tentative state beyond m; record m, u and stop status together for the next call.
>
> **Return:** owned requested outputs Y and the consumer's successor state.

The planner preserves contiguous row order, complete nonoverlapping partitions, and output-to-row identity. It makes no choice based on token values or observed output agreement.

2.4. Conditions for composition

Three distinct obligations connect the construction to Eq. 1.

1. **Planner structure.** Physical views cover the intended rows and extents, and arithmetic groups and tails cover each column once. The executor retains this row map in owned outputs and applies the consumer's accepted-prefix and pending-token semantics.
2. **Backend arithmetic.** Each used group and tail preserves the reference operands, quantization and reduction path, including descriptors, strides and placement. This is a backend condition, not a consequence of the partition algorithm. The Q8 instance requires the used one-to-four-column paths to satisfy it.
3. **Experimental checks.** Qualification compares requested full vectors and active state for specified inputs and boundaries. Online admission and diagnostic capture cover different parts of the implementation, as detailed in Section 3 and the supplement.

Channel packing introduces no reduction between groups: each output column is produced by its admitted inner path, and the final reshape is an index mapping. Given equal incoming producer state and the first two obligations, induction over the decoder dependency graph preserves the values consumed by later operations. Row-wise reconstruction preserves output identity; matching trim and pending-token semantics preserve the successor state. Repeating the argument across commit boundaries preserves continuation. Qualification supplies finite-case evidence for the backend condition rather than a universal native-kernel proof.

Full outputs and state are material to this argument. In a two-layer CPU diagnostic, a two-row call and two one-row calls agree on both argmax decisions, yet 242 of 256 output float bit patterns and 43 serialized state bytes differ; a subsequent forced continuation differs in 127 of 128 float bit patterns. The supplement gives the diagnostic and a second example in which changing the requested output set changes whether a consumer contract passes [5]. Token agreement cannot replace the vector and state premises.

3. Implementation

3.1. Two method instances and their controls

We implement the construction in modified llama.cpp paths [5]. The phase executor owns a resident target context with a dense, contiguous, nonwrapping sequence and F16 keys and values. A backend-neutral planner preserves ordered rows, physical extents and requested-output flags; native adapters supply numerical descriptors and operations. The CUDA instance uses causal, non-Flash attention for the supported Llama and Qwen architectures.

For Q8 projections, full arithmetic groups have $a = 4$. The packed path retains the existing small-column arithmetic [6], presents complete groups as channels, and restores column order afterward. The *backend-loop4* control already quantizes the wide activation once before invoking four-column and tail kernels. The comparison therefore tests execution organization beyond shared quantization. Cross-group packing requires at least eight known rows. In the $k = 3$ speculative applications, target verification has four rows, so this packing opportunity occurs in wider prefill work. The

SIMPLE/LOCAL factor separately controls placement of the physical attention cuts.

The 32B mixed Q4_K/Q6_K instance instead uses $a = 1$ and whole-graph cuts. B shares activation quantization and loops over scalar backend projections; C presents a $[K, 1, q]$ view and reshapes the result. Both retain the one-column dot-product specialization. The S3 C/B comparison consequently measures scheduling of independent scalar groups, distinct from the Q8 LOCAL-plus-packed combination. The weaker G control performs scalar slicing at graph level.

The Vulkan instance retains F32 activations where the scalar reference uses F32, together with the required attention extent and layout. It provides a separate implementation of the grouping principle, with its own numerical reference and controlled append measurements.

3.2. Output ownership, reuse and implementation coverage

After each native call, the executor synchronizes and copies requested vocabulary vectors into storage indexed by logical row. It then checks the physical append endpoint. This preserves an earlier accepted row when subsequent calls reuse native output buffers. Planning uses native metadata without reference replay or KV export.

Graph reuse compares the numerical policy, complete per-row extent vector, relevant head-normalization and rotary-placement choices, and the engine's topology and output-count conditions. Requested row indices are refreshed as inputs. A mismatch rebuilds the graph. The deployment fixes additional numerical modules and model choices; the frontend's cache-type tag alone does not encode them all.

Online admission checks the supported cache and output reserve. Marked backend-loop4 operators additionally check descriptors, actual device residency and dispatch constraints. Packed nodes use ordinary backend dispatch and do not inherit every marker check; their arithmetic evidence comes from deployment-specific qualification. Capture runs collect selected operators, full vectors and native state for offline comparison, while clean service timings exclude those observers. The supplement maps these mechanisms to the actual source paths.

A failed target group poisons its context. The published command-line worker contains other failures by terminating, rather than recovering a partially updated service transaction. Detailed failure coverage is in the supplement. The method's state-commit argument concerns successful admitted executions; a reusable service library would also need transaction-wide invalidation on failure.

4. Evaluation design

4.1. Questions, deployments and controls

The experiments follow the construction from numerical mechanism to service value. Captured width replays and attention-path repairs test why both groupings matter. The 32B study isolates scheduling across scalar arithmetic groups; the Q8 factorials separately vary projection packing and cut placement. Controlled appends and natural requests test how often local cuts remove useful work. Complete applications then account for draft execution and the work induced by a speculative policy.

The implementations run on an RTX A5000 with Linux/CUDA, an RTX 5070 Ti Laptop GPU with Windows/CUDA, and an Intel Arc B570 with Vulkan. Each deployment has its own reference. Resident model studies use a single dense, nonwrapping sequence, F16 keys and values, greedy generation, and non-Flash causal attention. The single-request A5000 panels use capacity 1024; the two-turn resident pilot uses 4096. Controlled Vulkan appends cover both capacities. Models come from the Qwen3, Llama and SmolLM2 families [7–9]. Table 1 maps each experiment to the method component it exercises. Frozen sources, qualification coverage and complete numerical inputs accompany the supplement [5].

In the Q8 panels, W and L denote SIMPLE whole-graph cuts and LOCAL attention cuts, while L4 and P denote backend-loop4 and packed projections. M denotes target-only generation, which loads no draft model. D4 denotes speculative generation with draft prefill microbatch four; the proposal count is $k = 3$. Thus, D4 is a draft setting, q counts target query rows, and Q8_0 is a weight format. Qwen targets use a Qwen3-0.6B draft; the Llama pair is Meta-Llama-3.1-8B-Instruct with Llama-3.2-1B-Instruct. All arms receive the same applicable residency, graph reuse and numerical fixes.

The 32B panel isolates scheduling across groups without a draft or a local-cut treatment. It fixes whole-graph attention cuts, two CPU threads, context capacity 1024, target microbatch and output reserve 32, and disabled CUDA graphs. Its 12 new roots comprise six coding and six mathematics tasks, with 39–187 prompt tokens and a 128-token output cap. Each of 144 fresh workers performs an identical-root warmup followed by one formal request. G slices scalar projections at graph level; B shares activation quantization and loops over scalar backend calls; C exposes those scalar computations as channels. C/B is the primary method comparison. Its $a = 1$ scalar-loop B is distinct from the $a = 4$ backend-loop4 control in the Q8 factorials.

The Qwen Q8 studies use pinned APPS and MATH inputs [10, 11], reusing task identities across sizes and platforms but establishing each deployment's reference separately. Llama's 12 roots were selected

Table 1
Method instances and the questions they test. Here a is the projection arithmetic-group width and q is the number of known target rows. A root is a task input; its repeated observations stay together in resampling. The Qwen Q8 service and factorial panels reuse the same 16 roots. Each row retains its own reference, experimental unit and clock.

Instance / question	Grouping and cut placement	Comparison	Units and measurement boundary
32B scheduling increment; A5000, mixed Q4_K/Q6_K	$a = 1$; whole-graph cuts fixed; target-only	G: graph scalar loop; B: shared quantization + backend scalar loop; C: independent scalar groups	12 roots × 4 rounds; 144 measured and 144 warmup requests; resident service
Q8 component attribution; A5000, Qwen3-8B/14B	$a = 4$; SIMPLE/LOCAL × backend-loop4/packed	Four-arm factorial; draft policy fixed; $q \leq 4$ in verification, wider prefill packs groups	16 roots × 4 balanced rows per model; 256 measured requests; resident service
Controlled boundary opportunity; B570, SmolLM2	Reference-compatible F32 path; $q = 2, 4, 32$; whole/local cuts	Matched crossing and inside appends	12 fixed cells, 3 paired repeats; prepared-state append, excluding state preparation
Complete Q8 application; A5000, Qwen3-8B/14B, Llama-8B	$a = 4$; full draft and target execution	Four target-only controls and two speculative paths; same-draft increment also reported	16/16/12 roots × 4 sessions; 384/384/288 measured requests; resident and worker clocks
Natural two-turn pilot; A5000, Qwen3-8B Q8	Capacity 4096; whole/local cuts; backend-loop4 fixed; no draft	Common canonical-history repair; local placement is the treatment	2 development conversations × 2 rounds × 2 arms; complete two-turn embedded service
Numerical migration alternative; RTX laptop, Qwen3-8B Q8	$q = 8$; old packed B versus modern U	Quantizer and projection contract change together	1 captured input, 12 pairs per boundary; extracted operator, excluding allocation, copies and checks

after excluding prior project exposure. The multi-turn pilot uses two development conversations from OpenAssistant [12]. These fixed workloads support deployment-specific estimates; model size is not a substitute for independent workload sampling.

4.2. Qualification and clocks

Offline qualification compares requested full vocabulary vectors and active KV entries in canonical logical order. Forced-prefix reference runs check speculative branches; an autonomous reference checks emitted tokens and stopping. Across qualification campaigns, the checked cases include prefill, continuation, boundary crossing, rejection, trim, pending tokens, termination and reset. Clean timing disables heavy observers and retains the corresponding endpoint, identity and execution-trace checks. The supplement associates observation scope with each implementation; it does not treat a clean endpoint check as a new full-state comparison.

Resident-service time includes reset, planning, target and applicable draft work, sampling, state bookkeeping, synchronization and requested-output copies. It excludes model loading, context construction and diagnostic export. The two-turn pilot additionally includes history rendering, tokenization, controller transport and durable per-decode accounting in its frozen host adapter. Worker clocks include initialization and primers and are reported separately. These are embedded request paths, without concurrent network serving or queueing. Prepared-state appends and extracted GPU-event operators have narrower boundaries, stated in their captions.

4.3. Estimands and uncertainty

For the 32B primary comparison, we average paired log time ratios over four rounds within each root and then equally over 12 roots:

$$\rho_{C/B} = \exp\left(\frac{1}{12} \sum_{r=1}^{12} \frac{1}{4} \sum_{s=1}^{4} \log \frac{t_{rsC}}{t_{rsB}} \right). \tag{4}$$

The paired-root bootstrap resamples six coding roots and six mathematics roots within their strata, preserving their equal contribution. It uses 20,000 draws and a 95% percentile interval for C/B; the two secondary G comparisons use 97.5% intervals. All repetitions and arms of a sampled root remain together. The ratio of pooled totals is a different estimand and is identified separately.

A5000 Q8 service and factorial effects use the ratio of total service time, $\sum_{r,s} t_{rsA} / \sum_{r,s} t_{rsB}$. Their 10,000-draw paired-root bootstrap uses 99% intervals for the five-comparison family. The interaction is an equal-root geometric ratio of component ratios. The factorials use four-arm Williams orders; six-arm service orders have partial balance, and the earlier 8B speculative pair retains its fixed order. Intervals condition on the observed sessions and tested roots. Three-repeat

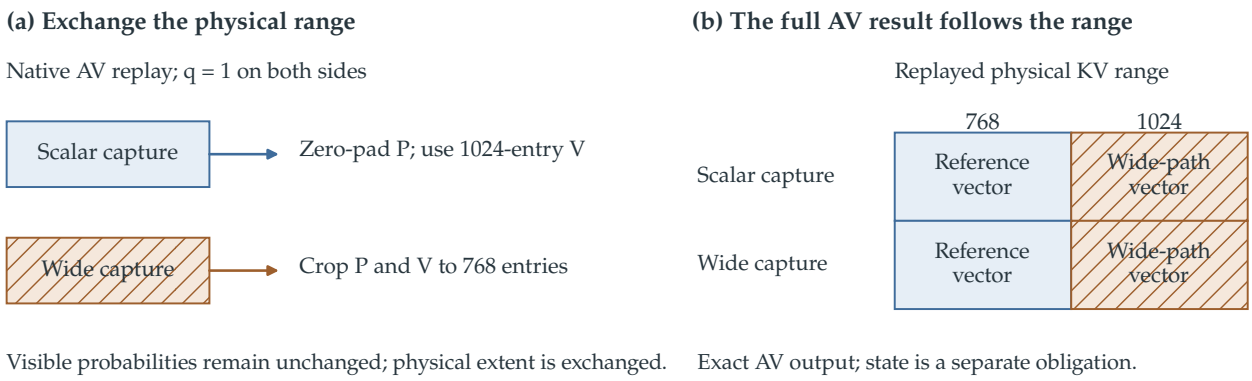


Figure 2: A physical-range intervention changes the attention–value result. The captured CUDA comparison fixes a one-query consumer and tests 768- and 1024-entry physical ranges. Cropping and padding reproduce the respective reference outputs while retaining the visible operands. The comparison covers the complete operator output; its boundary excludes persistent KV state and service time.

structural cells show their paired ranges; the two-episode pilot has no population interval. All time reductions are $100(1-\rho)$, not throughput increases.

The accompanying CPU scripts reaggregate the published rows and reconstruct the figures. Phase totals are descriptive sums of the same measured requests, kept separate from root-level inference. A supplementary leave-one-root-out check is explicitly post hoc and leaves the primary estimator and interval unchanged. ChatGPT (OpenAI, GPT-6 Astra Pro, accessed 13 November 2026) assisted manuscript preparation, CPU analysis and visualization code, and the conceptual diagrams. Numerical plots use the identified measurements; this revision adds no model execution.

5. Results

5.1. Physical range is part of the numerical problem

The CUDA diagnostic holds the query width at one and intervenes on the physical range supplied to attention–value multiplication. Cropping the wider execution from 1024 to 768 entries reproduces the shorter-reference output; padding the shorter execution to 1024 reproduces the wider output (Fig. 2). Visible operands and probability prefixes agree in the captured case, and the added probabilities are zero. The bidirectional intervention supports the physical-range mechanism at that operator. It observes the complete attention–value output, rather than inferring equality from the eventual argmax.

A historical Vulkan Llama companion demonstrates why arithmetic and attention requirements must hold together. Restoring the reference F32 projection path makes development endpoints agree, yet each speculative layout still differs in 99 of 128 committed full vectors on the long input. A joint attention repair removes these differences; disabling that repair in the same binary restores them [5]. This intervention changes physical extent, contiguity and resulting dispatch together. It establishes the need for the combined path in this case, while the CUDA crop/pad test isolates range at its captured operator. The supplement gives the repair controls and their full-vector scope.

At its tested boundaries, the 32B strong-control qualification passes comparisons of full 151,936-entry vocabulary vectors and active F16 KV across 64 layers. These observations complement the construction's conditional argument; the supplement identifies the coverage and retained records for the other profiles.

5.2. Independent groups improve on shared quantization

Figure 3 shows the strongest direct service comparison for the execution organization. On the 32B target-only deployment, C/B is 0.984864, with a 95% paired-root interval of [0.982148, 0.987721]. The corresponding reduction is 1.51% [1.23%, 1.79%]. All 48 measured paired requests favor C, but the inferential unit is the 12 roots, not the 48 repeats.

The weaker control explains why baseline choice matters. B/G is 0.956855 and C/G is 0.942372. Moving the scalar loop into the backend and sharing quantization already obtains much of the advantage over G. C/B isolates the remaining organization change.

The phase account localizes that change. Prefill falls from 143.927 to 137.984 s, saving 5.942 s across 48 requests; decode changes from 216.663 to 216.690 s. Known prompt rows can expose independent scalar groups, whereas the target-only decode proceeds one token at a time. The measured service totals are 364.832 and 358.925 s, a net saving of 5.907 s. These same-panel sums locate the benefit without separating quantization, kernel and launch time. Their pooled ratio is 0.983809; the primary root-weighted result remains Eq. 4. This is the scheduling increment of the

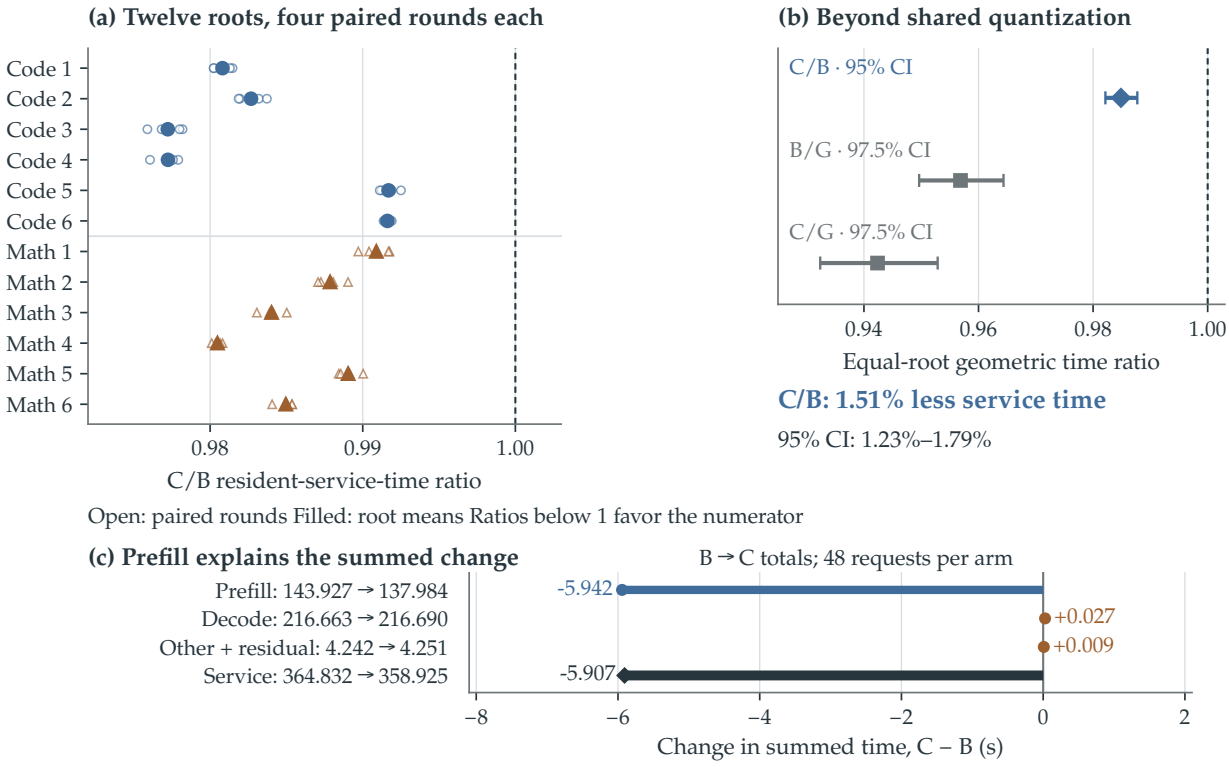


Figure 3: Independent scalar groups improve the strong 32B control, with the saving concentrated in prefill. A5000, Qwen3-32B-Q4_K_M, target-only, $a = 1$, whole-graph cuts. Root points average paired log ratios over four rounds; the primary C/B interval is a 95% bootstrap over 12 roots, stratified by coding and mathematics. Secondary G comparisons use 97.5% intervals. B shares activation quantization before its backend scalar loop; C schedules those scalar groups independently. Phase panels show descriptive sums over the same 48 measured requests per arm, in seconds; they are separate from the root-weighted estimate. The resident clock includes reset and output ownership and excludes loading and diagnostic capture.

$a = 1$, whole-cut instance, distinct from the local-cut and four-column packing treatments below.

5.3. Packing and local cuts have different opportunities

The Q8 factorials fix the draft policy and cross cut placement with projection organization (Fig. 4). Packing reduces service by 1.64% with SIMPLE and 1.56% with LOCAL at 8B, and by 0.99% and 0.80% at 14B. Their backend-loop4 controls already share activation quantization, so these contrasts measure the remaining packed-group increment.

Local cuts reduce service by 0.21% and 0.30% at fixed loop4. At fixed packing, the reductions are 0.13% and 0.11%, with both intervals containing zero reduction. The equal-root interactions are 1.000696 and 1.001703, indicating slight attenuation when both components are applied. At $k = 3$, target verification has at most four queries, so it does not enter the multi-group Q8 packing route. Execution records place that route in prefill. The components consequently address different opportunities even within one request.

5.4. Boundary geometry explains the local-cut results

Service value depends on how often useful opportunities occur, how much work each one saves, and what organization or speculative work is added. We examine these terms within each experiment's own contract. Opportunity counts, prepared-state timings and live-work counts illuminate different terms; they are not fitted into a cross-platform performance prediction.

In the natural 8B factorial, 20 of 2,224 logical verification cycles require an additional whole-graph split. SIMPLE uses 2,244 graphs and LOCAL 2,224 for the same target rows: just 0.899% of cycles present that opportunity. The controlled B570 appends make the opportunity explicit (Fig. 5). A crossing starts one row before a physical boundary and forces the whole-graph split $1 + (q - 1)$, for $q = 2, 4, 32$. Across capacities 1024 and 4096, local execution takes 20.34–44.68% less append time in these cells. Matched inside controls take 0.43–3.74% more time. Three paired observations per cell show repeat variation, not a workload confidence interval.

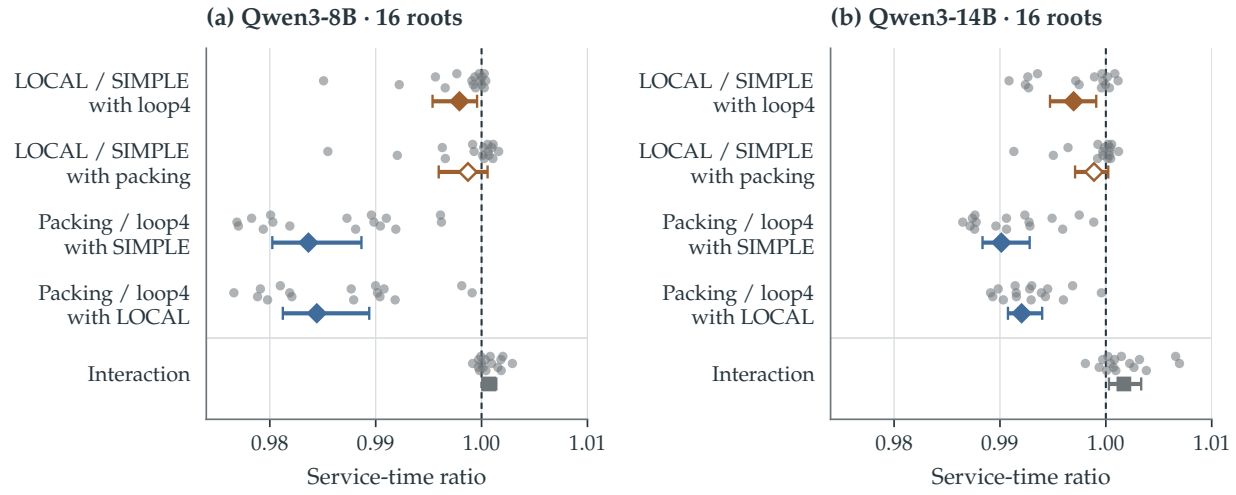


Figure 4: Separating the two execution changes. Balanced A5000 Q8 factorials fix draft prefill microbatch four and proposal count three. Faint points show individual root ratios; summary markers show ratios of total resident-service time with 99% paired-root intervals from the five-comparison family. Projection contrasts hold cut placement fixed; cut contrasts hold projection fixed. The interaction summary is an equal-root geometric ratio of ratios. The two models share 16 task roots and are separate measurements.

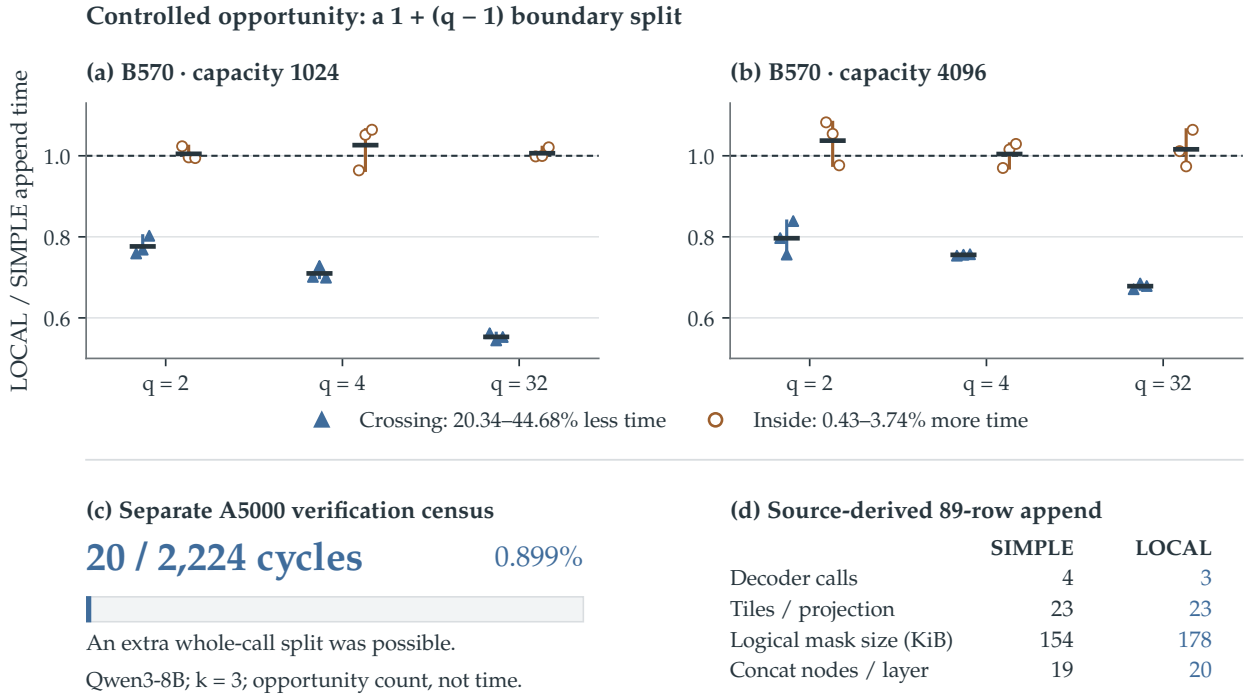


Figure 5: Opportunity size, frequency and remaining work. B570 panels compare LOCAL/SIMPLE prepared-state append time; ranges are the minimum and maximum of three paired repeats per cell. Their clock includes binding, decode, synchronization and requested-output copying, excluding state preparation. Capacity and prefix region change together. A separate A5000 panel reports 20 cut opportunities in 2,224 verification cycles, a cycle frequency rather than a fraction of service time. The 89-row panel is source-derived: calls decrease while the 23 ordered tiles per fully consumed projection remain equal and logical mask/concatenation work increases. Those counts and tensor sizes are not measured launches, traffic or latency. Earlier $16 + 16$ reversals remain in the supplement.

Table 2
Natural multi-turn pilot: LOCAL/SIMPLE complete two-turn service ratios. The original backend-loop4 projection, capacity 4096 and request protocol are fixed; there is no draft. Two rounds repeat each development conversation. These four pairs are descriptive, with no population confidence interval.

Episode	Round 1	Round 2
Development conversation A	1.0997	1.0024
Development conversation B	1.0886	0.9888
Pooled ratio		1.04220

Crossing alone is not sufficient. Earlier $q = 32$ cells with a $16 + 16$ whole-graph split include cases favoring SIMPLE. In the natural multi-turn pilot, the four paired episode ratios are 1.0997, 1.0024, 1.0886 and 0.9888 (Table 2). Across the eight timed episodes, LOCAL takes 37.921 s versus 36.385 s for SIMPLE, a descriptive ratio of 1.04220. The pilot therefore provides no observed local-cut service improvement; two development conversations cannot establish a population ranking.

The two-turn producer explains why history reuse does not supply a separate LOCAL advantage. Its generation prompt contains a four-token empty-think suffix. The next canonical history omits that suffix before the assistant body, so the body's token positions and conditioning change. In both actual free responses, all 128 body token IDs are retained by the parser, yet exact prefix reuse stops before the obsolete suffix. Both methods trim there and reconstruct the same canonical continuation. Earlier, already canonicalized history can remain cached on later turns; that reuse is shared. The supplement gives the token timeline and distinguishes these observations from supplied-history predictions.

The pilot's complete calls fall from 1,072 to 1,070. Of the 1.535 s pooled service difference, 1.420 s lies in the common CPU sampling span, and the equal-partition control also slows. This account locates the observed difference without attributing it to local attention. A separate source-derived 89-row example makes the remaining opportunity precise: whole calls $[24, 32, 32, 1]$ become local calls $[32, 32, 25]$, while both retain the same 23 ordered projection tiles and 89 input rows. The local builder creates larger rectangular masks and one more concatenation node per layer. Saving an outer call can therefore leave the main arithmetic unchanged and add assembly work. This source analysis explains competing costs; the proposed append was not timed.

5.5. Application gains are larger than the method increment

In all three A5000 Q8 application panels, the complete speculative path reduces resident-service time relative to each of the four target-only controls. Against the fastest measured fixed M, LOCAL-packed D4 reduces resident service by 17.21% for Qwen3-8B, 25.54% for Qwen3-14B, and 11.52% for Llama-8B. Their adjusted 99% intervals are [11.17%, 24.61%], [18.21%, 34.89%], and [4.33%, 20.95%], respectively. These comparisons include the draft and the complete verification application. Against SIMPLE-loop4 with the same D4 policy, the corresponding increments are 2.01%, 1.08% and 1.56%. The earlier 8B same-D4 pair retains a fixed order; its execution contrast is less isolated than the balanced factorial. Complete root distributions, intervals and all fixed controls appear in the supplement.

Initialization can change the decision. For the 14B request burst, worker time including loading, integrity checks and primers falls from 662.546 s for W-P-M to 594.559 s for LOCAL-packed D4. Llama's resident requests improve, yet its short worker workload favors target-only execution: the fastest M worker takes 355.730 s versus 368.822 s for LOCAL-packed D4. The earlier 32B panel instead favors ordinary grouped generation; grouped D4 takes about 14% more resident time. A B570 pilot likewise favors target-only generation despite fewer target graphs in the speculative path. The supplement retains these complete cost accounts.

5.6. Efficient wider calls can induce more application work

A Windows RTX equal-work diagnostic makes one $q = 8$ call 3.94–11.91% cheaper than two $q = 4$ calls from the same state with the same eight known tokens and requested outputs. In live generation, increasing proposals from $k = 3$ to $k = 7$ instead raises service time by 13.58–14.59% across the four methods (Fig. 6). For the same 11,472 emitted tokens, verification cycles fall from 3,504 to 2,296, but verified rows rise from 14,016 to 18,368 and draft proposals from 10,512 to 16,072. The policy saves cycles while generating more target and draft work. Its complete cost outweighs the improvement demonstrated for a fixed eight-row operation.

5.7. A modern backend is a different engineering choice

A recent upstream llama.cpp few-row implementation [13] provides a useful alternative. On one captured Qwen3-8B Q8 input to a 4096×4096 projection with $q = 8$, the ratio of upstream U to the extracted legacy packed Q8 operator B is 0.612924, with a 95% paired-repeat interval [0.607906, 0.618665]. This reduces extracted-operator time by 38.7%. U uses a different quantization and projection contract: 32,766 of 32,768 output bit patterns differ from the legacy reference.

U supplies a faster extracted operator when the numerical reference can change. Full default-backend qualification and paired service timing have not been run. The legacy-compatible C and C1 candidates take 1.1264 and 1.0261 times B's complete-operator time in their separate sessions. The supplementary variant

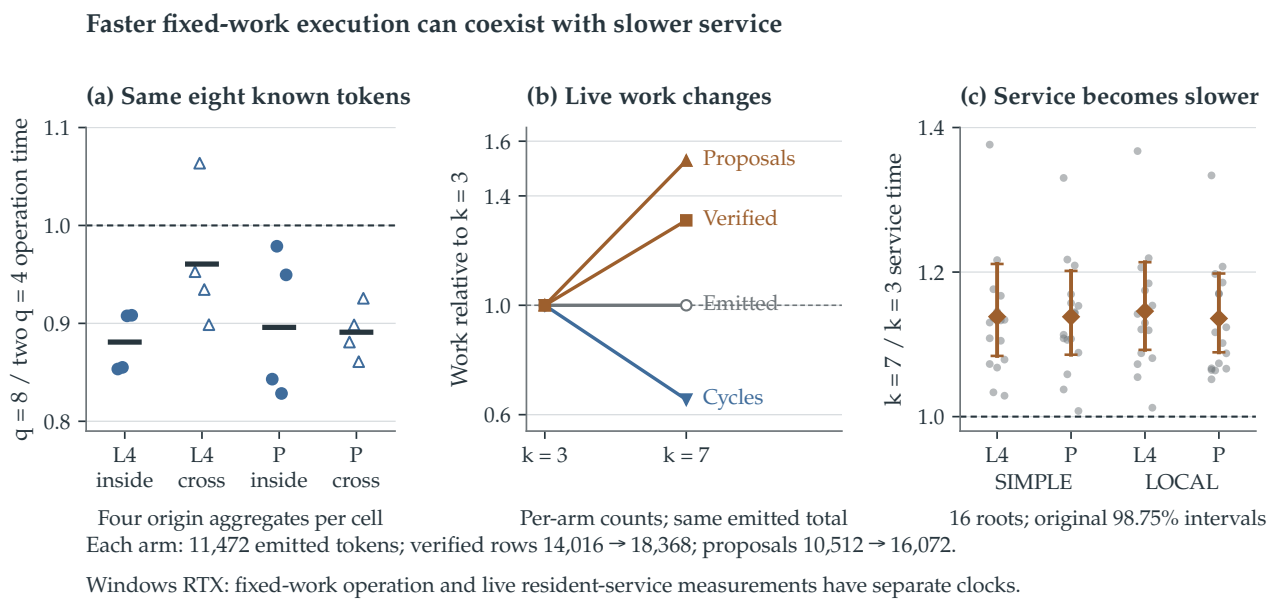


Figure 6: Wider execution and more speculation are different interventions. (a) Equal-work RTX operations hold the eight known tokens, incoming state and requested outputs fixed; points are four origin-level aggregates per cell. (b) Live-work counts cover 128 measured requests per arm (16 roots in eight repeated order blocks). The four methods share the displayed counts at each k. (c) Dots show the 16 root ratios; diamonds show ratios of total service with the original 98.75% paired-root intervals. L4 means backend-loop4 and P packed projection. The operation and resident-service clocks belong to separate experiments.

table identifies the changes, resources and timing boundaries of these alternatives.

6. Related work

Matching an existing numerical reference. Yang et al. reconstruct the arithmetic of cuBLAS kernels and enforce bitwise behavior through compiler transformations and static verification [4]. Their computation descriptors separate reduction structure from scheduling: output tiling and pipelining can change when each output retains its arithmetic. Their reconstruction targets matrix multiplication; attention receives a theoretical treatment whose implementation remains open. Our construction adds query-specific physical KV views, local attention cuts and successor-state obligations within a quantized decoder.

Batch invariance and fixed scheduling. He and Thinking Machines Lab construct batch-invariant normalization, matrix multiplication and attention [1]. Their attention analysis shows why separating cached KV entries from newly computed entries can change reduction grouping. Fixed reductions and the numerical significance of cache boundaries are therefore established ideas. Here, the selected reference supplies each query's physical extent, including initialized entries outside its logical causal prefix; the executor retains that extent where the attention computation requires it.

CoRun fixes the execution context through isolated prefill, padded fixed-shape decode, position-invariant kernels and request-specific sampling state [2]. That policy preserves repeatable request-local arithmetic while allowing co-runners and batch slots to change. Our input is a previously selected numerical deployment, which can prescribe different physical ranges for adjacent queries. Local cuts honor those differences while surrounding compatible work remains grouped.

Outputs, persistent state and repair. Vosti specifies deterministic inference through request outputs and verifies engine invariants together with kernel contracts [14]. Its KV invariant covers live and retained reusable entries, tying each value to its logical token prefix. Its engine proof depends on imported kernel contracts, which a separate verifier discharges for supported kernels. Our conditional argument uses output and state obligations to compose an execution transformation in an existing implementation; finite qualification supplies the empirical coverage reported here.

LLM-42 permits dynamic batching on a fast path, then uses fixed-shape verification to commit a consistent prefix or roll back [3]. Even when tokens agree, it replaces the fast path's KV entries with verifier-produced entries. Our transformation instead preserves the selected target path before consumer commit. It adds no separate numerical repair pass; ordinary speculative token acceptance still belongs to the consumer.

Quantized kernels and inference optimization. The implementation inherits deterministic quantized

operations and four-column execution from `llama_reproducible` [6]. Modern `llama.cpp` uses integer tensor-core operations, asynchronous copies and reuse across column groups in its few-row quantized matrix multiplication [13]. These are existing optimization mechanisms. Our evaluation separates reorganization under the selected reference from adoption of a modern operator with a different quantizer and projection arithmetic. FlashAttention reduces attention memory traffic through tiling [15]; FlashInfer supports heterogeneous KV layouts and load-balanced execution [16]. Fixed-reference compatibility adds an implementation-level numerical condition to those optimization goals.

Speculative decoding. Speculative decoding and sampling verify several draft tokens together to amortize target work [17, 18]. Their distribution-preservation arguments concern the sampling algorithm. Liu et al. adapt verification effort to candidate uncertainty [19]. Our method organizes already selected query tokens under a numerical reference. We therefore separate complete speculative-application results from factorials that fix the draft policy and isolate projection packing, local cuts and their interaction.

7. Discussion and limitations

7.1. Compatibility and numerical migration

Regression testing, exact replay and unchanged consumers motivate retaining an existing numerical interface during optimization.

The 1.51% reduction in the 32B experiment quantifies an execution opportunity that remains after the control already shares activation quantization. Independent scalar-group scheduling improves this fixed-reference implementation without changing the arithmetic contract. A deployment that permits a different reference can instead adopt modern arithmetic, as the faster extracted upstream operator illustrates.

7.2. What the construction transfers

The reusable method is to identify the physical and arithmetic requirements separately, place each cut at the operations that require it, and preserve the requested outputs and successor state. The one-column mixed-quantization and four-column Q8 implementations demonstrate distinct instantiations. Existing small-column kernels and packing techniques provide the components; their arrangement around query-specific cache ranges and the consumer contract provides the method studied here.

Transfer requires backend work. Equal extents can hide different descriptors, and equal shapes can select different quantizers or reductions. Paged or sliding caches, Flash attention, mixed-device partitions and concurrent sequences would require new view and state obligations. The reported implementations and finite qualifications cover the stated dense-cache profiles. A persistent service must also provide recovery across its complete state-changing transaction; the current worker terminates on failures outside its target-group invalidation boundary.

7.3. Scope of the performance evidence

The measured benefits concentrate in known-row work. S3 saves time in prefill while decode remains nearly unchanged. In the Q8 applications with $k = 3$, target verification supplies at most four rows and therefore does not enter multi-group packing. Local cuts depend on boundary opportunities and on how much surrounding work survives the cut: sparse crossings limit the natural-request increment, while unchanged tile counts and added mask assembly can consume a saved outer call. Wider speculation adds another cost by creating more target and draft work.

Several studies reuse development inputs, and the Qwen sizes share root identities. Root-clustered intervals describe the stated workloads and sessions; they do not sample cross-day or cross-machine variation. Controlled cells and the two-conversation pilot remain descriptive. The post-hoc root-deletion check tests sensitivity of the S3 point estimate, while the prespecified result and its interval remain primary. Source-derived tile and mask counts describe possible work, whereas actual occupancy and memory traffic remain unmeasured.

8. Conclusion

A fixed numerical reference leaves freedom to organize parallel work across groups that retain their internal arithmetic. Separating physical KV ranges from arithmetic groups places attention cuts where they are required while keeping compatible surrounding computation grouped. Row-owned outputs, accepted-prefix trimming and pending-token bookkeeping preserve the information needed by subsequent consumers under the stated backend conditions.

On the evaluated A5000 Qwen3-32B deployment, independent scalar-group execution reduces target-only resident-service time by 1.51% relative to a strong same-contract backend scalar-loop control that already shares activation quantization. This instance uses $a = 1$ and whole-call cuts; its saving is concentrated in prefill while decode remains nearly unchanged. The component and application studies explain how service value emerges: compatible grouping opportunities must occur, the retained surrounding work must yield a saving, and that saving must exceed the additional organization and speculative work.

Data and code availability

Implementation sources, public timing rows, numerical projections and reconstruction scripts are linked by fixed revisions in the supplementary material and accompanying source manifest [5]. The source package includes the inputs and scripts needed to reproduce the reported CPU statistics, tables and figures. Some underlying native logs and tensor records are company-restricted; selected temporary intermediates were pruned and are no longer available. Those records are not included in the public reconstruction. Original model weights are identified at their sources and are not redistributed with the paper.

Declaration of generative AI and AI-assisted technologies

ChatGPT (OpenAI, GPT-6 Astra Pro, accessed 13 November 2026) assisted manuscript drafting and English editing, literature and code checks, CPU statistical analysis, visualization code, conceptual diagrams, and LaTeX layout review. Numerical figures were generated by scripts from the identified experimental records. Project records also report AI assistance in the earlier research-code workflow. During this revision, no model or GPU measurements were added, and the recorded experimental data were preserved.

References

[1] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. *Thinking Machines Lab: Connectionism*, September 2025. doi: 10.64434/tml.20250910. URL https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

[2] Shiju Zhao, Jiacheng Yang, Qihang Chen, Junhao Hu, Jiaqi Zheng, Guihai Chen, and Xusheng Chen. CoRun: Padding is simple and efficient for deterministic LLM inference. arXiv:2608.14376v1, 2026. URL https://arxiv.org/abs/2608.14376v1.

[3] Raja Gond, Aditya K. Kamath, Ramachandran Ramjee, and Ashish Panwar. LLM-42: Enabling determinism in LLM inference with verified speculation. arXiv:2601.17768v2, 2026. URL https://arxiv.org/abs/2601.17768v2.

[4] Ziteng Yang, Nicholas J. Riasanovsky, Warren Deng, and Vivek Sarkar. Taming bitwise behavior in GPU kernels with tensor core: Black-box reconstruction, compiler enforcement, and static verification. arXiv:2609.11356v1, 2026. URL https://arxiv.org/abs/2609.11356v1.

[5] leecdiang. Implementation and companion evidence for fixed-reference parallel inference. GitHub repository, 2026. URL https://github.com/leecdiang/HeteroSpec. Experiment commits, evidence locations, and access scope are listed in the supplementary material.

[6] Andrew Miller. llama_reproducible: Deterministic quantized inference implementation. GitHub repository, 2026. URL https://github.com/amiller/llama_reproducible/tree/f0dafdbd9dba40bd0e596f74f3da5f592900fca4. Commit f0dafdbd9dba40bd0e596f74f3da5f592900fca4; accessed November 13, 2026.

[7] An Yang et al. Qwen3 technical report. arXiv:2505.09388v1, 2025. URL https://arxiv.org/abs/2505.09388v1.

[8] Aaron Grattafiori et al. The Llama 3 herd of models. arXiv:2407.21783v3, 2024. URL https://arxiv.org/abs/2407.21783v3.

[9] Loubna Ben Allal et al. SmolLM2: When smol goes big—data-centric training of a small language model. arXiv:2502.02737v1, 2025. URL https://arxiv.org/abs/2502.02737v1.

[10] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. arXiv:2105.09938v3, 2021. URL https://arxiv.org/abs/2105.09938v3.

[11] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. arXiv:2103.03874v2, 2021. URL https://arxiv.org/abs/2103.03874v2.

[12] Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi-Rui Tam, Keith Stevens, Abdullah Barhoum, Nguyen Minh Duc, Oliver Stanley, Richárd Nagyfi, Shahul ES, Sameer Suri, David Glushkov, Arnav Dantuluri, Andrew Maguire, Christoph Schuhmann, Huu Nguyen, and Alexander Mattick. OpenAssistant conversations—democratizing large language model alignment. arXiv:2304.07327v2, 2023. URL https://arxiv.org/abs/2304.07327v2.

[13] llama.cpp contributors. Few-row quantized matrix multiplication in llama.cpp. Source code, ggml/src/ggml-cuda/mmq-few.cu, 2026. URL https://github.com/ggml-org/llama.cpp/blob/0fb8f157702702f33f4e018de8701b2569a1bc28/ggml/src/ggml-cuda/mmq-few.cu. Commit 0fb8f157702702f33f4e018de8701b2569a1bc28; accessed November 13, 2026.

[14] Jianxing Qin, Alexander Du, Danfeng Zhang, Matthew Lentz, and Danyang Zhuo. Vosti: Specifying, implementing, and verifying deterministic LLM inference. arXiv:2609.38981v1, 2026. URL https://arxiv.org/abs/2609.38981v1.

[15] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. arXiv:2205.14135v2, 2022. URL https://arxiv.org/abs/2205.14135v2.

[16] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. arXiv:2501.01005v2, 2025. URL https://arxiv.org/abs/2501.01005v2.

[17] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

[18] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318v1, 2023. URL https://arxiv.org/abs/2302.01318v1.

[19] Siran Liu, Yang Ye, Qianchao Zhu, Zane Cao, and Yongchao He. HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12930–12947. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.acl-long.589. URL https://aclanthology.org/2026.acl-long.589/.